Technische Hochschule Nürnberg

Fakultät Informatik

When types prevent bugs: Exploring Rust's safety benefits in filesystem development

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Florian Meißner

Matrikelnummer: 3210376

Erstgutachter: Prof. Dr. Hans Löhr
Zweitgutachter: Prof. Dr. Michael Zapf

Table of Contents

Chapter 1

Introduction

Since the beginning of computer programming, there has been a discrepancy between the input states an interface formally accepts, and the input states that are sound to handle. For example, a reciprocal function $f(x) = 1/x$ might formally accept a 32 bit integer — and therefore all of its $2 \hat{} 32$ input states —, but the mathematical formula it tries to model will not give a sensible result for $x = 0$; least of all if the function in turn returns an integer, since there is no integer n: $1 / x = n$.

One straightforward solution has always been to limit the function domain via documentation. Users of that function are expected to read that documentation and recognize that it is a violation of interface contract to call it with $x = 0$. Violation of that contract would in turn result in an error, a crash, or — worse yet — Undefined Behaviour. An approach with drawbacks, as there are now two sources of truth about the function domain. One of these — the function signature, expressed in code — is already technically incorrect, as we have not stated a way to express "integers without zero" as a valid type (Listing 1). Additionally, as the program developes, the chance of both sources of truth to get further out-of-sync increases. For example, special behavior could be introduced to handle the undefined case (see Listing 2). This discrepancy, between the high-level contract, expressible only in additional information in text form, and the function signature the compiler handles, raises the following question: Can we encode this precondition in such a way that the function signature makes it impossible to pass in values that violate any API contract?

```c
// `n` CANNOT BE ZERO!
double reciprocal(int n) {
  if (n == 0) {
    crash(); // we warned you!
  }
  return (double) 1 / n;
}
```

Listing 1: A typical C implementation of `reciprocal()`

```c
#include <math.h>
// `n` CANNOT BE ZERO!
double reciprocal(int n) {
  if (n == 0) {
    println("Division by 0 is not allowed!");
    return NAN;
  }
  return (double) 1 / n;
}
```

Listing 2: `reciprocal()` with a changed contract.

In programming languages where we have strong typing (see Section 2.1.2), we can enforce invariants about types we create, since we have to explicitly provide the methods of construction for values of these types, and if any invariants are not upheld, we can detect this unsoundness and trigger an error. This allows us to express function signatures of the kind discussed previously, by creating a new type `NonZeroI32` that represents the idea of an integer that cannot be zero. Unfortunately, not all languages provide the features necessary to formulate such powerful types. One prominent example is C, which is predominantly used in systems level programming, both in general and in the domain we are looking at in this work. Besides the basic datatypes that exist primarily to differentiate between CPU directives — integers, floating points, characters, pointers — users can create composites of these types via the `struct` or `union` constructs, and define functions to work on these datatypes only. But these type requirements can easily be subverted, because in C, casting between different types is often implicit, and there are no mechanism to enforce invariants on a type — all user code that can "see" a struct can create or delete instances as it sees fit.

That is why, in this work, we will look at Rust: a modern language that is gaining rapid traction in the area of systems programming, and has a strong, expressible type system as one of its flagship features. This enables us to explore the latter approach. Listing 3 shows such an implementation in Rust. It utilizes the concept of a "newtype struct": a `struct` with one anonymous member, which is used to wrap this inner element and to effectively give new type semantics to it. If the inner value is chosen to be private, ergo not visible or accessible from code from a different module, this prevents any alternative way of constructing or modifying values of this new type other than what the module creator chooses to provide. This achieves exactly the desired effect. By making the inner value private, we then ensure that users of our library are forced to use the only construction method we provide them with, `NonZeroI32::new(i32)`. This `new()` function can be total, since it returns a result value representing a fallible computation. In that sense, the function signature of `new()` expresses that **every 32-bit integer is either a valid non-zero 32-bit integer or an error**.

```rust
pub struct NonZeroI32(i32);

impl NonZeroI32 {
  pub fn new(n: i32) -> Result<Self> {
    if n == 0 {
      Err("n == 0 is not allowed!")
    } else {
      Ok(n)
    }
  }
}

pub fn reciprocal(n: NonZeroI32) -> f64 {
  // ...
}
```

Listing 3: A new type `NonZeroI32` that represents the idea of a 32-bit integer **guaranteed** to not be zero

This is one of several features of Rust that promise improving soundness checking and language expressibility with no or minimal runtime impact. System programming is an area where these soundness guarantees are especially important. While a logic error stemming from an unchecked invariant in an application can crash this application or corrupt its data, a kernel or Operating System (OS) bug can cause those failure states in any and every subsystem and program. Faulty OS behaviour can cause instability on every layer of the system, and endanger the work of all users. Simultaneously, system programming has to produce performant instructions, since they will be running as the backbone of the OS, which often leads to complex data structures where keeping invariants as cognitive load, for programmers to check upon every code change, is unlikely to produce said stability in the long run. That is also why solutions that penalize compile time only are especially valuable, because compile time is often expendable — after all, most OSs run many times as often as they compile.

Our work is therefore targeting a subsystem of modern OS development. Filesystems were chosen, because they fulfil all stated criteria: their performance matters, their correctness is crucial for stable system usage, and they deal with complicated, highly optimized data structures where invariants between those structures play a major role. Additionally, H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu [2] found that filesystem-related kernel modules represent a valuable target for improvements through Rust because of their high ratio of bugs per lines of code. Kernel module development is not a trivial task, though, and challenges during code writing and testing would cost additional time. Debugging is also comparably harder when the targeted module is injected into the OS itself. FUSE (**F**ilesystem in **USE**rspace) offers a way out for this conundrum [3]. It is a framework that works by combining a kernel module with a userspace library. Filesystems using it link against the userspace library and use it to communicate with the kernel module, which will redirect OS requests to the userspace program. This provides an environment not unlike a

sandbox: the kernel module is well known and tested, and approximately bug-free. The concrete filesystem, which we will implement, will live in userspace and therefore be easy to debug and not critical to system stability. And since the APIs are quite similar, approaches found to be working when modeling FUSE filesystems will with high probability also work in general, e.g. as native kernel modules.

In the following thesis, we will introduce the chosen tools Rust (Section 2.1) and FUSE (Section 2.2), followed by a comparison to similar works (Chapter 3). A methodology is introduced (Chapter 4), and the implementation thereof described in detail (Chapter 5). Then we evaluate our solution against a sample of CVEs (Chapter 6) and draw our conclusion (Chapter 7). Finally, we discuss possible future work (Chapter 8).

[4]

Chapter 2
Background

In the following sections we will describe tools Rust and FUSE, and the features they bring that make them suitable for our thesis.

2.1 Rust

Rust is a modern programming language, created in 2006 by a Mozilla employee and endorsed by Mozilla in hopes of creating a safer web browser engine. It is intended for use as a low level systems language, usually competing with C in benchmarks [5]; compared to C, it incorporates numerous improvements aimed at increasing safety and automatic correctness of written software, with minimal to no runtime impact [6].

2.1.1 Unsafe Rust

2.1.2 Type system

[7], [8]

[9], [10]

Rust's type system, or rather the specific properties thereof, are an important factor for its usability in correctness-centered software. We will focus on two important characterizations of a type system: static vs. dynamic, and strong vs. weak. We define (as seen in B. Liskov and S. Zilles [11]) a strong type system as a system where every function that expects a certain type of parameter will only accept parameters of that type. In other words, the less implicit type conversions are performed by a type system, the stronger it is deemed. In contrast, weak type systems perform many implicit conversions. Static type systems, as previously defined in B. Liskov and S. Zilles [11, p. 474f], have the ability to calculate types of all values during compilation, and can therefore statically verify type correctness. In dynamic type systems, the concrete types of values are indeterminate during compile-time and the compiler will emit the necessary runtime checks that result in execution errors when a type mismatch happens.

We value the strength of a type system highly, because it provides us with the foundational guarantees on which the higher-level contracts can be expressed. For example, given we want to enforce that a file handle be in a specific, well defined state (open, writable) before writing to it, with a weak type system, we would have two possible scenarios for inconsistency: the file handle implicitly converting into another type, which potentially does not check those contractual assertions, and a value of another type implicitly converting into a file handle, where the conversion itself possibly circumvents some checks. Additionally, strong type systems can prevent bugs of a certain class, where the wrong value is passed into a function, if that value has a different type. The weaker the type system, the more implicit conversions may accidentally trigger, which would make the program compile without catching the type error. This is not to say that all implicit conversions are inherently bad, but they pose certain risks and are usually hindering when trying to reason about program code, since every possible place where an implicit conversion could take place exponentially adds possible states that must be considered. On the other hand, implicit conversions can lead to more concise code and can reduce boilerplate, which, when applied correctly, can improve readability and ability to reason about code effects. Ergo a middle ground should be found.

Rust takes a strong design stance in that regard [12, ch. 10.7]. The language philosophy is to use as little implicit conversions as possible, to limit the amount of "surprises" encountered when reasoning about code effects. This is a design principle that aligns well with our goals.

The aspect of static vs. dynamic type checking is also of importance for our thesis. Dynamic type checking would bring two disadvantages in this case. First, type errors are reported not during development, but during actual code execution. This not only delays the bug finding process, but also requires that every part of the program be executed during testing to ensure correctness. Second, since we aim to enhance type checking to also maintain our higher-level invariants, this comes at the cost of additional logic. Whereas static type checking would run this logic only during compilation, dynamic type checking leads to runtime overhead, which is usually a critical property of system code, to be reduced whenever possible.

Rust is a predominantly statically typed language, with optional dynamic typing (see The Rust Project Developers [12, 10.1.15]) by the name of "trait objects". This is consistent with other high-level languages, like C++ [13, section "Type"]. This gives us the flexibility to fall back to dynamic typing whenever necessary, while having a rich toolset available for expressing type constraints in a static, compiler-verified fashion.

2.1.3 Borrow checker, lifetimes and ownership

One of the core improvements made by Rust in the area of static correctness is its memory model, centered around ownership, static reference tracking and reference lifetime [10].

In Rust, values are, by default, moved instead of copied. In fact, making a type copy-able involves implementing a special trait (`Clone`), while moving values is core to the type system and automatically available for every type. In contrast, types are memory-copied by default in C++, and must manually influence or prohibit that behavior [13]. Moving values was introduced in C++ 11 and has to be enabled manually. This core difference, called "linear types" in type theory, brings a range of benefits to the type system [14]. Since types can be "consumed", ergo moved into a consuming function without being returned, these consuming functions or methods can be used to implement resource management that would otherwise be difficult to do correctly. For example, a file handle can be closed, which would render any I/O operation on it ineffectual and would usually lead to an error. In languages where values are copied by default or can be copied without limitations, this creates inconsistencies: If the handle is copied, then one copy closed, how should the second copy behave? How can a user of the API guarantee that the handle is not used after closing, if it could have been copied a number of times? This problem can be managed by manually implementing checks on the handle type, usually with the help of shared memory and synchronization. Alternatively, if the language provides ways to limit arbitrary copying, these limits can be carefully implemented. But the default still leaves room for errors and makes the copy case more ergonomic by definition. In Rust, these questions do not arise, since the file handle type would simply not implement the `Clone` trait and every function with a value parameter of `FileHandle` would automatically consume this parameter, rendering it unusable in the caller site.

Besides ownership, Rust provides references as a handle to a non-owned value that can be used to access it [12, ch. 10.1.13]. There are two types of references: mutable-exclusive and immutable-shared. This hints at a core principle in Rust's memory model, namely that shared mutable state is prohibited — without further safeguards and encapsulation. This eliminates error classes like data races (see Section 2.1.4) and iterator invalidation [15], and makes the code easier to reason about by not letting references spread out over modules influence local state. To guarantee that references do not exist longer than the data they are referring to, Rust uses lifetimes, which are part of the type of a value but can often be inferred by the compiler without writing extra syntax. The borrow checker — a part of the static analysis tools inside the compiler — then validates through these lifetimes that references are only "alive" when their pointee is "alive", and that the invariants regarding mutable and shared references are held up.

2.1.4 No data races and fearless concurrency

Concurreny is an important aspects of today's software. Single machines usually have several physical processor cores, and programs wanting to take advantage of the hardware capabilities need to deal with some variant of concurrency. Furthermore, even not hitting hardware limitations, modern software usually consists of multiple distinct subroutines — rendering a GUI, receiving events, working calculations, providing an API — that are desired to run simultaneously, lest responsivity and latency suffer, and with it the user experience. This is also true for OS code and filesystems, as these too must take advantage of modern computer architecture to deliver

performance goals. A filesystem on a typical modern computer can expect many different programs to concurrently access different files, or even use the filesystem for inter-process communication. This makes managing concurrency scenarios mandatory. For example, BtrFS spawns many worker threads to handle the background work necessary to maintain the filesystem structure [16]. Concurrency, while ubiquitous, brings many implementational challenges in modern languages. C, for example, declares every data race UB [17]. Rust differs from these in that it leverages the type systems to make certain guarantees in concurrent programming, namely that data races cannot occur.

Data races happen when data is shared mutably between threads without proper synchronization, mutably meaning more than one thread has the means of modifying the state [18], [19]. Since execution order of instructions between multiple threads is undefined, when multiple threads attempt to change the shared data at the same time, the outcome — the state of the shared data — is also undefined. This arises simply from the fact that reads and writes from the different threads can be interleaved arbitrarily and nondeterministically by the processor, resulting in operations that seem atomic or sequential to the programmer being interrupted or reordered.

Listing 4 shows a simple case of a data race. Picture the `counter()` function running concurrently in two threads. Each instance has the goal of increasing the global counter by one, a million times. Yet, as we execute this program multiple times, different values will be reached, ranging from 1,000,000 to 2,000,000. This is due to operation reordering: since an increment is internally a sequence of “read value” -> “increase” -> “write value”, when these instructions get interleaved, multiple threads interfere with each others incrementation, leading to loss of writes.

```
static mut COUNTER: usize = 0;

fn counter() {
  for i in 0..1_000_000 {
    unsafe {
      COUNTER += 1;
    }
  }
}

pub fn main() {
  let t1 = std::thread::spawn(&counter);
  let t2 = std::thread::spawn(&counter);

  println!("{COUNTER}");
}
// ...
```

Listing 4: Excerpt from the `readdir` trampoline, passing the Rust vector of directory entries into the C filler function.

Since data races can lead to program states that are hard to reason about, automatic prevention would be beneficial to the development process [12]. By encoding the "concurrent-safe-ness" of data types in the type system, Rust brings about such a mechanism. There are two traits that function as markers for these properties: `Send` and `Sync`. `Send` signals that variables of this type can safely be transferred between threads, while `Sync` declares a type is safe to be accessed simultaneously by multiple threads. These traits contain no members, since their only function is marking thread-safety. Manually implementing them requires an unsafe block, since it is up to the programmer to check that the type really is thread-safe; on the other side, types only consisting of `Send`/`Sync` members can have their `Send`/`Sync`-ness derived automatically via a macro.

As a side note it should be mentioned that while safe Rust guarantees freedom of data races, code races or deadlocks can still happen. We assume that these cannot be prevented in sufficiently powerful languages, as that would require checking every possible combination of thread execution states, which is NP-complete in recursion to the halting problem.

2.1.5 Error handling

One of the strong points of Rusts stance on a strong and flexible type system comes with its choice in error handling. One common pitfall with C, that is the cause of many production bugs, is that C uses patterns to signal errors that are implicit, usually reserving part of the range of the function for error codes, and sometimes storing additional info on the error in a global variable — commonly `errno`. This handling is implicit because, since the return type does not inherently encode the possibility of an error, manual checks by the programmer must be inserted for every potential source of errors, which increases cognitive load. Forgetting to insert such a check almost always leads to unsoundness, since a potential error will be handled as some sort of valid success value. Circumstances are even more difficult when dealing with `void` functions, that do not return a value but rely on side effects instead. Since no return value handling is required for "normal", non-error code paths, the chance of forgetting to check for errors is increased again. Although some C compilers define custom extension attributes that can annotate a function to emit a warning when the function's result is unused[1], no standardized solution exists. [20]

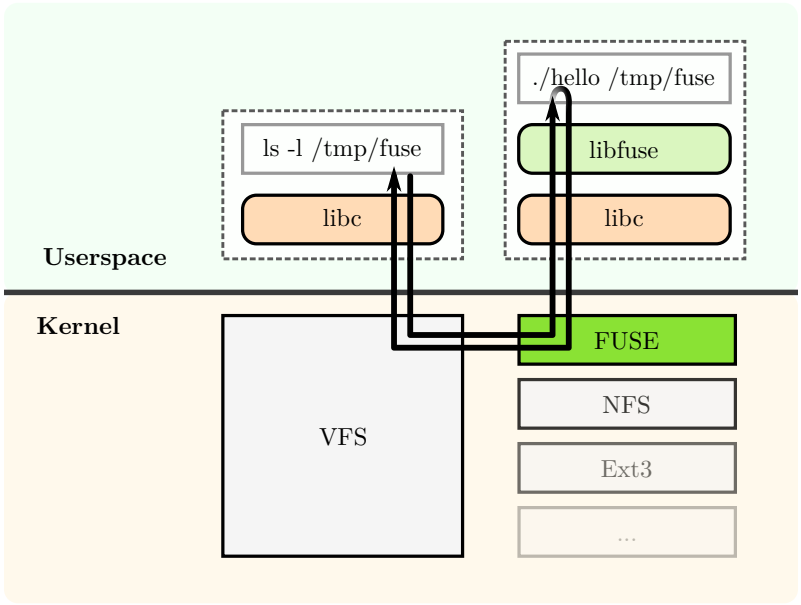
Rust solves this issue by combining the strong type system guarantees with sum types, or tagged unions. Unions, as they are used in C, are usually only useful in implementing low-level data structures or protocols where memory efficiency is of grand importance. Since they require the programmer to keep track of which data type is stored inside them at any moment — or, to manually track this state via extra variables — variable accesses of invalid type are encouraged, often leading to undefined behaviour. Rusts decision to introduce labelling leads to strongly-typed unions — called `enum`s in Rust — that are always checked, by nature of the type system. The programmer is forced by the compiler to handle every occuring case explicitly, and where dynamic handling is required, runtime checks are emitted at the least.

[1] https://gcc.gnu.org/onlinedocs/gcc/Common-Attributes.html

Rust then introduces the "result type", `enum Result<T, E> { Ok(T), Err(E) }`. This represents a tagged union two type parameters: the success type `T` and the error type `E`. With this, a function from `A -> B` can trivially be transformed into a fallible function `A -> Result<T, E>` with error type `E`. `A -> Result<T, E>` can never be accidentally be misused as `T`, nor `E`: the type system enforces the values be unpacked and every case handled accordingly. Therefore, there is no need to encode error states in numeric return values, such as the Linux kernel using negative return values for error signaling. This must be considered a hack, as it severely limits the design space of functions, and diverges program semantic from the specified data types.

Coupled with Rust's `#[must_use]` function annotation, which is set on the STD `Result` implementation, Rust prevents accidentally not handling error results, whether by ignoring a return value or by using a fallible result.

2.2 FUSE



[2]

FUSE (**F**ilesystem in **USE**rspace) is a mechanism for filesystem modules in the Linux kernel, introduced in `2.6.14` in 2005 [3], [21]. It lifts the requirement of filesystems being kernel modules, which meant they run in kernel space and therefore have the highest priviledge level, which potentially impacts security and stability of the system. Additionally, installing a new kernel module involves a great amount of additional work: At best, the module has to be compiled via Dynamic Kernel Module Support (DKMS) against every kernel in use by target machines. At worst, a copy of the kernel source code has to be maintained with the filesystem module added, and on every upstream kernel update the whole kernel has to be recompiled. Kernel modules also operate

[2] https://commons.wikimedia.org/wiki/File:FUSE_structure.svg

under a strict subset of available tools and resources, and are more complex to program, which increases development costs [22]. Being able to write filesystems as generic userspace programs can therefore yield various benefits.

The FUSE project replaces the need for a distinct kernel module per filesystem by providing a general module, which the userspace daemon then communicates with. The necessary kernel APIs can therefore be forwarded via a message socket, solving the need for elevating filesystem code to kernel mode. A more familiar experience is provided through the libfuse C library (libfuse), which is a C library presenting an array of functions and types with an API similar to that of the kernel. This abstracts away the message passing style of the daemon, and enables users to simply implement the functions established in the the POSIX standard (POSIX) filesystem section — `read`, `mkdir`, `ioctl` and similar. This creates the illusion that implementing a filesystem is nothing more than providing a set of function bodys for the filesystem API inside `libc`, which takes away much of the complexity behind the scenes and enables creating simple filesystems with low development cost and entry barrier.

Chapter 3
Related work

The following chapters enumerate similar projects, as well as their similarities and differences in research goal, design and evaluation.

3.1 Bento

S. Miller *et al.* [22] propose a problem space similar to our work: filesystem development is tedious, because kernel modules are hard to debug; a filesystem kernel module crashing the system through bugs is also suboptimal. Still, in their evaluation FUSE falls short because of significant overhead introduced into metadata-heavy workloads — such as `git clone`[3] of big repositories — which incentivized them to develop a novel solution. Bento is a framework for developing Linux filesystems that plugs into the Virtual File System layer (VFS) kernel component and provides a sandbox for filesystem implementations written in Rust. Modules using this framework can be loaded, reloaded and updated without interrupting userspace work, and have their crashes isolated from the rest of the kernel, increasing resilience to filesystem bugs.

While this approach would bring benefits also to our project, it is our estimate that it would have increasse development effort manyfold without significantly increasing the value and validity of our results, which is why we decided against it.F Furthermore, their evaluation consists only of benchmarking their implementation against the C-native `ext4` module, with no security criteria considered, whereas our work focusses on evaluating security benefits with no significant consideration of performance criteria.

3.2 `rust-fatfs`

S. Oikawa [23] present a Rust reimplementation of the FAT filesystem family created by Microsoft. FAT (File Allocation Table) was originally created in 1977, with the newest variant — `FAT32` — initially published 1996. Despite that, it stays relevant until today, as it is often used in technically constrained environments, where it is preferred for its comparatively simple structure

[3] https://git-scm.com/docs/git-clone

and implementation. For this reason, it is relied upon by digital cameras for their dedicated storage cards, and as filesystem for the boot partition in the UEFI specification [24, p. 462], [25], [26].

`rust-fatfs` implements the `FAT12`, `FAT16` and `FAT32` members of the FAT family. It is implemented as a kernel module, whereas our library lives in userspace with the usage of FUSE. The authors were motivated by lack of existing filesystem kernel modules written in Rust, as well as a general interest in Rust given its recent rise in popularity regarding the linux kernel, and its promising safety features. `rust-fatfs` was solely evaluated from a performance perspective, with the authors measuring the number of instructions executed during sample workloads, run inside a VM. These metrics are compared to the established C kernel module. We did not consider empirical performance analysis, and focus on which kinds of CVEs can be prevented.

3.3 `fuser`

The `fuser` crate, maintained by GitHub user `cberner`, is currently the most used FUSE bindings crate on `crates.rs`, counting over 150k downloads per month, and over 1k stars on GitHub [27]. It is not an academic project, but a practical solution for integrating FUSE into the Rust ecosystem, thereby leveraging the advantages of Rusts architecture for filesystem development. While our goals are not conceptually different as a whole, there is no explicit focus on security. Types are modeled very closely to the underlying C architecture, although users are not forced to create C ABI compatible functions, since that — like in our solution — is abstracted away. In fact, probably due to increased performance, `fuser` does not use libfuse but talks directly to the FUSE module via socket. This is a complex, low-level undertaking and was out of scope for our project, although it would have been interesting in principle, as the same design principles could still be applied. Still, results are comparable because the crate models `fuse_operations` very closely to libfuse. Also, while we chose to use the high-level libfuse API, which identifies files via paths directly and submits results synchronously, `fuser` leverages the low-level (LL) libfuse API, which uses Inodes to represent entries and message helper structs to return asynchronously. This was carefully considered, but the high-level API simplifies some implementation parts while losing almost none of the targeted design space for safe systems programming, so it was deemed the better choice.

[27]

3.4 Rust for Linux[1]

Rust for Linux[4] represents a major effort to allow a second high-level language into the linux kernel, next to C [28]. The language's promise of memory-safe low-level code with many additional features — such as a strict type system, sum types and ownership model — that are beneficial to correct and performant software, is also cited as motivation. Since December '25, the Rust

[4] https://rust-for-linux.com/

subsystem has been promoted to “core” from “experimental” regarding the Linux kerner — it is now a first-class citizen [29]. Currently, the subsystems developed with or in Rust include several filesystems such as `tarfs`, `PuzzleFS` and a read-only `ext2` implementation[5] [2].

H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu [2] evaluate the Rust-for-Linux project w.r.t three categories: security enhancements, performance impact, and improvements in the development process such as code readability and project attractiveness for junior developers. They conclude that while Rust does seem to partially improve the areas it promises to, high-level logic errors are still very much possible, and certain pitfalls can lead to vast degradation of performance. They also conclude that, based on their metrics of bugs per lines of code, filesystems and filesystem-adjacent modules like the `ext4` driver and the Linux block device subsystem would be lucrative next targets. This aligns with our assessment.

[5] https://www.phoronix.com/news/Rust-VFS-Linux-V2-Now-With-EXT2

Chapter 4
Concept

First, we study works that explore safety aspects of Rust features, especially regarding its type system. We also study similar works to this thesis that leverage Rust to write safety-focused system code. We extract principles and patterns, which can then be applied to constructs and invariants encountered when developing an API for filesystems.

To limit the scope of this work, we filter the list of needed FUSE operations to provide an API for a minimal file system. For each operation deemed necessary, we enumerate a list of contracts and invariants that are either explicitly or implictly expressed in code or documentation. We then decide for each invariant, if and how it can be incorporated in our system of checks, prioritizing static analysis and compile-time correctness over automatically emitting runtime checks. For each invariant that is regarded impossible to check in this sense, we document the shortcomings of our system and potential improvements for the future.

The CVE (Common Vulnerabilities and Exposures) system is an internationally accepted, de-facto standard, cataloguing system for vulnerabilities FIXME. In colloquial terms, “a CVE” refers to an entry in the CVE database, managed by MITRE. For evaluation, we collect a sample of recent recorded CVEs from the Linux kernel filesystem subsystem. This e.g. includes the VFS and several shipped filesystem implementations. We utilize a search mask for the National Vulnerability Database[6] for filtering the relevant data. Each CVE from the sample is then analyzed and sorted into three categories:

🟢	🟡	🔴
Could have been prevented using our method	Could partially have been prevented / could have been prevented under certain circumstances	Could not have been prevented

We include reasoning for every categorization, and if deemed non-preventable, we document what could be improved in our method, or why the problem is out-of-scope for our thesis.

[6] https://nvd.nist.gov/

Chapter 5
Implementation

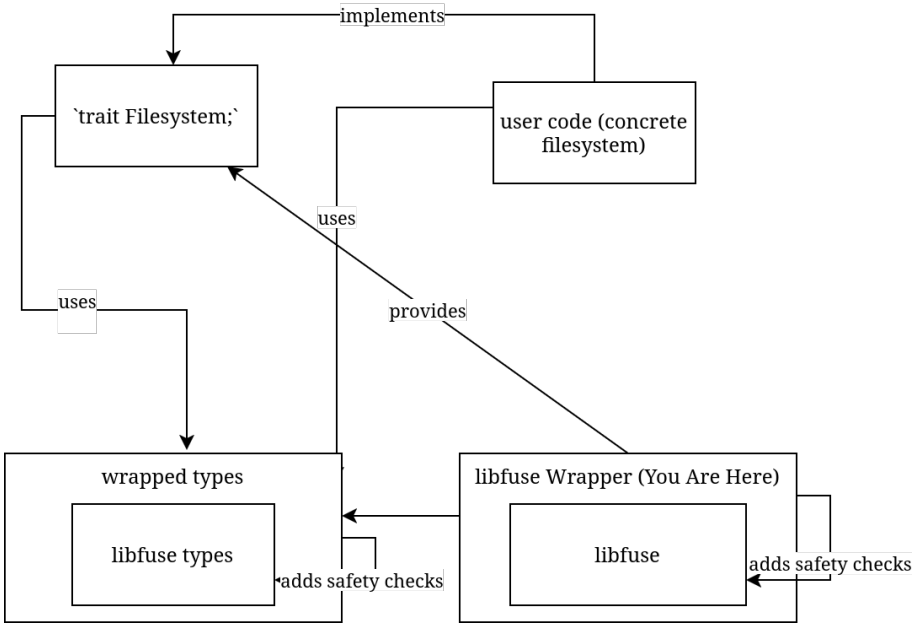


Figure 1: High level architecture overview of our wrapper library.

Figure 1 shows the architecture we chose for our project. Wrapping around generated bindings to the underlying C code of **libfuse**, it exposes a number of Rust equivalents for relevant data types. These higher-level models include correctness checks on all properties where that is feasible, therefore significantly decreasing the surface for errors on the filesystem implementor's side, which is the main goal of our thesis. As central interface, a `Filesystem` trait is provided, which closely models the underlying `fuse_operations` as a collection of relevant callbacks which a filesystem must provide, and which in turn constitute the main functionality of that filesystem. The `Filesystem` trait achieves these advantages by utilizing the constructed higher-level type mappings.

The following sections present a detailed elaboration of the concrete structure of our implementation, as well as the conditions and limitations exacted by our chosen frameworks. We first establish the areas of Rust that are of central relevance to our design, and the challenges they impose. This is followed by justified descriptions of the data structures we modeled and the design pattern we chose, as well as justifications why we chose them.

5.1 Basic C interop

Excluding internal compiler errors, safe Rust can never cause UB in the resulting binary program. In unsafe Rust, this is not the case; the programmer now has to uphold several invariants to ensure soundness. Unlike in the C standard, where behavior in any situation not explicitly defined by the language standard is implicitly "undefined", Rust limits these invariants to a set of specific, well-documented cases [12, ch. 17.2], [17]. This makes reviewing the soundness property of unsafe code easier.

5.1.1 Pointers

Regarding use of raw pointers in unsafe Rust, the following invariants exist:

1. No dereferencing of dangling or unaligned pointers. This is obvious, since these requirements stem from the underlying hardware, and C also declares them invalid [12, ch. 17.2], [17].
2. Respect aliasing rules: no pointer is allowed to point to memory that is also pointed-to by a mutable reference, since a mutable reference in Rust is guaranteed to be exclusive. This is needed to provide same of Rusts safety guarantees: data races, use-after-free errors and iterator invalidation are all inherently impossible without shared mutable state.
3. Respect immutability: no pointer is allowed to modify data that is also pointed-to by a shared reference, since a value behind a shared reference is guaranteed not to change. This also follows naturally from the basic type system axioms in Rust: if a reference pointing to a value is held, then by rule 2 it is guaranteed that no mutable reference to this value exist simultaneously. Therefore the value must not change.
4. Values in memory must be valid for their respective types: pointers must not be used to change the representation in memory of to a value — or reference — to a state which is not valid for the type this value — or reference — has. E.g. a `NonZeroU8`, represented in memory as a `u8`, will have one bit pattern that would correspond to a numeric zero and is therefore illegal. This rule is also emergent, following from the basic type system principle that variables of a type must hold values of exactly that type. Circumventing this would completely subdue the advantages arising from Rust's powerful type system.

Because libfuse calls all our callbacks with at least one C pointer, we have to check these invariants as rigorously as possible before we call into user code, if we want to eliminate them as sources of UB.

1. We have to differentiate between three cases:
   - **Unaligned pointer**: this is easy, as Rust provides `ptr::is_aligned()`, which takes a pointer and detects misalignment.
   - **Dangling null-pointer**: this is also easy, both manually and through the Rust-provided `ptr::is_null()`, which takes a pointer and detects null-ness.

- **Dangling non-null pointer**: this happens when a pointer is used-after-free or if pointer arithmetic goes wrong, and is much harder to avoid. Since we do not control memory allocation in C, we largely have to trust C code to not pass us pointers from this category. This would cause UB and should therefore be documented visibly as soundness assumption.

2. For pointers passed to us by libfuse, the solution is simply to not create a reference to it. If it is necessary to pass a mutable reference into user code, an intermediate owned value must be created, and the target value must be copied in and out of that intermediate.
3. When dealing with non-const pointers, care must be taken to not create a shared reference to it. Const pointers do not matter in that aspect, since it is impossible to modify values through them, given they are not cast to non-const pointers.
4. This only matters when primitive C-style casts or `mem::transmute()` are used, as otherwise the Rust typesystem protects us from writing values of the wrong type, even inside unsafe blocks. Writing to a pointer can involve writing raw bytes; if that is required, extra care must be taken, and it is therefore usually better to avoid this.

5.1.2 Strings and Unicode

Rust's native string types (`str`, `String`) exclusively store UTF-8 [30, ch. 8.2]. The main kind of strings this library needs to handle are the file paths that filesystem callbacks are called on. The encoding of those is platform-dependent, usually being C-like ASCII strings on Unix-like systems and UTF-16 on newer Windows versions. Correctly detecting and handling string encodings is a hard problem , and since UTF-8 is a superset of ASCII, we chose to not handle UTF-16 or other cases and emit an error when encountering non-UTF-8 input. This limits the complexity of the prototype without limiting the scope of the reseach question.

5.1.3 Unwinding across FFI boundaries

When a Rust program is compiled with stack unwinding support and a panic is triggered, the default unwind handler will walk up the stack in order to react to the panic, collecting debug information or cleaning up data [12, ch. 14]. In a program using FFI, this can lead to crossing into another language runtime while walking the stack. Doing so correctly is a non-trivial task and "creates unique opportunities for undefined behavior, especially when multiple language runtimes are involved." On the other hand, turning unwinding off causes helpful stack traces and debug information to be lost when a panic occurs. We therefore decided to keep unwinding behaviour while preventing any panic from propagating across an FFI boundary.

Every function that is visible to C can potentially be called from an environment where unwinding works differently — or not at all. Therefore each of those functions must be panic-free. As of now, there is no compiler flag or lint that detects or prevents use of panicking functions, operators or language keywords. As a result, this requirement must be checked manually by reviewing the

source code of the functions in question, and, recursively, the functions they call. A convention exists to note possible panics in a section of the function documentation, but even the standard library does not consistenly follow it.

While preventing panics in self-maintained code requires careful manual analysis, this is not possible for user-provided functions. For this, there exists a function `panic::catch_unwind()` [31] that takes a closure and executes it, catching any unwind that would occur and returning an error instead. Wrapping the call to user code inside this function ensures that no panic will be propagated up the call stack from this point on Listing 5.

```
fn call_into_user_code<FS: Filesystem, T>(
    method: &str,
    user_fn: impl FnOnce() -> Result<T, Errno>,
) -> Result<T, (String, Errno)> {
    let fs = std::any::type_name::<FS>();
    std::panic::catch_unwind(core::panic::AssertUnwindSafe(user_fn))
        .map_err(|panic| {
            // abort, since internal state of filesystem impl can now be inconsistent
            state::clear();
            (
                format!("PANIC on `{fs}::{method}`:\n\n{panic:?}\n"),
                Errno::ENOTRECOVERABLE,
            )
        })
        .and_then(|inner| inner.map_err(|e| (format!("Error in user code `{fs}::{method}
`"), e)))
}
```

Listing 5: An exemplary trampoline function signature implementing compile-time static dispatch via generics

5.2 FUSE operations

The libfuse operations struct contains 43 callbacks to implement, most of which are optional and not needed for a filesystem to work properly. If we can forgo modifying the state, and create a read-only filesystem, the number of required calls can be brought down to 3 [32, p. structfuse___operations.html], [32, p. example_2hello_8c.html].

Some of the operations which are superflous for our experiments include:

- `lock`/`flock`: These are used for file locking, enabling safe concurrent access, and locking primitives across processes. Since our filesystem is readonly, no locking is needed.
- `ioctl`: Needed for special I/O commands, when simple seeking to byte offsets, and reading/writing from them is not sufficient. Examples include ejecting CD-ROM or rewinding data tape. Not needed for our general-purpose minimal filesystem.

- `write`/`sync`/`fsync`: These are used for writing data out, and to force flushing buffers to the underlying storage. Irrelevant in a read-only filesystem.
- `mkdir`/`link`/`create`/`mknod`/`unlink`/`rmdir`: Creating and deleting of entries of various types. Not relevant for a read-only filesystem.

All implemented operations check their pointer arguments for validity, with the methods discussed earlier (Section 5.1.1). The obligatory `path` argument, that identifies the entry to operate on, is converted from a C string into native Rust, and also checked for validity (Section 5.1.2). After basic correctness of inputs has been ensured, the code tries to load the filesystem object from the global registry. This is done to minimize unneccessary work when some inputs are not sound. The load could fail, e.g. in case the user code triggered a panic earlier, or due to a bug in the wrapper library. This is also handled (Section 5.3).

After that, some operation-specific instructions are executed, and a context is set up to call into user code without triggering panic unwinding (Section 5.1.3). libfuse datatypes are converted to our Rust representations, adding the implicit safety checks. After the user code has been executed, the results are converted back into libfuse types, and success of the operation is signaled up the stacks.

Next up is a detailed description of the required calls, accompanied with their respective C and Rust signatures.

5.2.1 getattr

```c
int(*   getattr )(const char *, struct stat *, struct fuse_file_info *fi)
```

```rust
pub unsafe extern "C" fn getattr<FS: Filesystem>(
    path: *const i8,
    stat_out: *mut libfuse::stat,
    _fuse_file_info_out: *mut libfuse::fuse_file_info,
) -> i32
```

This provides information about a filesystem entry, be it a file, a directory, a symbolic link, or some sort of special device. Without this call, no metadata could be queried about our filesystem, and its usefulness would be severely limited.

This function, as do all of the callbacks described here, takes a path in form of a C string to describe the filesystem object to query. It also takes two additional parameters: a `stat` struct as output, to be filled with the resulting metadata (Section 5.4.2), and an optional `fuse_file_info`, which may be set when the file in question is currently opened, and can provide additional metadata and FUSE settings if set.

We chose to mostly ignore the `fuse_file_info` for now, as it is only used under specific circumstances, and even then provides only very specialized attributes that would go beyond the scope of this exploration. The other two parameters are checked as per concept.

It is the users job to create an instance of `struct Stat` and pass it back to us. This newtype struct contains a valid `stat` fuse struct inside, which is needed as return value written into the output pointer argument of same name, and can trivially convert to one.

5.2.2 readdir

This function's job is, given a directory as path, provide a list of child entries, enabling directory content listing (as e.g. in the `ls` command). An offset can be provided to support partial listings over multiple calls, however we chose to ignore this, as is allowed in the documentation [32, p. structfuse___operations.html].

There exists an alternative, more complex mode, which we could have chosen to support: Implement the additional `opendir` operation to open the directory to enumerate as a file descriptor. Then, `readdir` is called on the active file descriptor, providing a view of the directory that is guaranteed to be the same as when `opendir` was called. This was deemed unneccessary to explore the given research questions, and skipped subsequently.

The API is designed, so that the `readdir` implementation does not just return, or write into, an array of entries. Instead, a function pointer to a "filler" function is provided. Our operation has to call this function for every entry. Some of the filler functions parameters correspond to entry metadata, others, like a pointer to an opaque data buffer, have to be forwarded Listing 6[32, p. structfuse___operations.html].

```
// ...
for entry in entries {
    let entry_as_c_string = try_errno!(CString::new(entry.clone()).map_err(|e| {
        // convert Rust string to C string
    }));
    debug!(?path, "filling entry '{entry}'");
    let fill_result = unsafe {
        filler_fn(
            data_ptr,
            entry_as_c_string.as_ptr(),
            ptr::null(), /* setting `stat` struct to NULL, as per `hello.c` */
            0,           /*: offset */
            libfuse::fuse_fill_dir_flags_FUSE_FILL_DIR_DEFAULTS,
        )
    };

    if fill_result != 0 {
        // propagate errno
    }
}
// ...
```

Listing 6: Excerpt from the `readdir` trampoline, passing the Rust vector of directory entries into the C filler function.

5.2.3 read

The `read` operation provides us with the means to fetch the content of files, complementing our set of operations to obtain a usable, if minimal, filesystem. It takes as parameters a size and an offset, determining the range of content to be read, as well as a C character array to store the data in. Again, an optional `fuse_file_info` struct pointer is provided, which we can ignore.

Dealing with the size and offset parameters provided a challenge. While the requested size parameter is of type `size_t`, which is usually defined as 64-bit integer on modern systems to be used for indexing memory, we have to return the actual amount of read bytes as signed 32-bit integer. As such, we can only signal successful reads of up to 2^31 - 1 bytes. Further investigation showed that maximum read size is limited by the Linux kernel [33], so the limitation in the API seems reasonable, assuming other operating systems impose similar limits. We therefore have to carefully check if the input parameter is inside the allowed range, and convert between the respective integer types, in addition to the usual checks. Furthermore, Rust does not provide a native method of specifying an upper bound for a vector length as part of the type signature, so manual checks are required after the user code returns the data. Dependent types or refinement types would be a possible solution, but are not available in Rust aside from research projects [34].

If the size checks pass, a pointer copy is issued, for which Rust STD provides a function. Because we use `unsafe`, we documented the assumptions made and invariants we checked, as is common practice (Listing 7).

```
  // Safety: we checked that the buffer is big enough to hold the returned data (if
`size` argument was correct).
  //         Also we checked that the pointer is aligned and non-null.
  //         Also, the areas cannot overlap, since the Rust vector has reserved its own
memory.
  unsafe {
      ptr::copy_nonoverlapping(
          result.content.as_ptr(),
          buf as *mut u8,
          result.content.len(),
      );
  }
```

Listing 7: Excerpt from the `read` trampoline, copying the read result into the provided C buffer.

5.3 Initialization and Global State Management

The libfuse initialization routine takes a struct of callback function pointers (`fuse_ops`), which creates the following problem: since the C signature is predetermined, user functions cannot be used, because that would force signatures of user functions to use the lower-level C types which we try to avoid. That means, even though there is a one-to-one correspondence between FUSE operation callbacks and trait methods on the `Filesystem` trait, they are not compatible and cannot be used interchangably. The obvious approach is to provide **trampoline functions**, which then wrap, transform and safety-check the C type values on call and dispatch into user code. A non-trivial problem, one that is not obvious at first sight, is how the trampoline knows which filesystem implementation to dispatch to. There are two basic options how to use the trampolines:

1. Use one global trampoline per callback, and somehow transport the choice on which filesystem to use inside the C arguments that **our libfuse wrapper** gets passed by **libfuse**.
2. Somehow generate a set of trampolines per user filesystem, which are then hard-coded towards the specific filesystem implementation.

A way to implement option 1 is provided in the form of a `void *private_data` pointer that can be passed to **libfuse** during filesystem registration. This pointer can contain arbitrary user-specified data, and is not used by **libfuse** except for making it available to every fuse operation via the `fuse_get_context`[7] function.

Since it is possible to store a Rust pointer inside a C void pointer, **our libfuse wrapper** can submit a pointer to the user implementation as payload for `private_data`, then let each trampoline poll

[7] https://libfuse.github.io/doxygen/fuse_8h.html#a5fce94a5343884568736b6e0e2855b0e

the FUSE context struct, cast the void pointer back to a trait object reference and dispatch into the corresponding trait method. This has the following disadvantages:

- Decaying a managed Rust reference into a raw pointer loses the advantage of lifetime tracking that is one of Rusts fortes in the undertaking of creating safe systems-level code.

Manual care has to be taken not to invoke a use-after-free, accessing an uninitialized or unauthorized memory location or — in the best case — simply leaking memory. In fact, the safest option would be to initialize this data pointer once, and then never free it, since it is the dealloc part that introduces memory unsafety to a system, and even if leaking memory (and not calling destructors) is acceptable, since libfuse passes around non-const pointers to everything, bugs at any point of both our trampolines and libfuse can easily lead to access of corrupted pointers and therefore to UB. This is usually a tradeoff that must be accepted when dealing with FFI into unsafe languages, but should be mitigated whenever feasible.

Both disadvantages would in theory be prevented by a solution after option 2. Fortunately, with the use of generics, Rust brings includes the tools to implement such a solution. As seen in Listing 8, this exemplary trampoline function is generic over types implementing our `Filesystem` trait. This leads the Rust compiler to generate a concrete, independent `getattr` trampoline function for every trait implementation of `Filesystem` that is used to call our initialization function. The generic approach is then combined with a singleton registry[8] which provides a global map of values, indexed by types. We can now store the concrete user-supplied filesystem struct inside this registry and use the type of this filesystem struct as index, which additionally will be deduced implicitly by the compiler from the argument types of our initialization function. That means, given there are no other implemenations of our `Filesystem` trait in scope when declaring the generic functions, the type system guarantees us that the user's type is the only one that can be used for dispatching, shielding even against potential programmer oversight.

This has the drawback of only allowing one instance of a concrete `Filesystem` type to be mounted per process. But since — if needed — newtype structs can be used to create different concrete types with minimal boilerplate, this was deemed tolerable.

```
pub unsafe extern "C" fn getattr<FS: Filesystem>(
    path: *const i8,
    stat_out: *mut libfuse::stat,
    _fuse_file_info_out: *mut libfuse::fuse_file_info,
) -> i32 {
```

Listing 8: An exemplary trampoline function signature implementing compile-time static dispatch via generics

[8] https://crates.io/crates/singleton-registry

5.4 Type modeling

Creating thin high-level representations of the low-level data types that make up the libfuse API, that nonetheless verify as many correctness properties as possible, is the main focus of this project. Where feasible, these properties are checked during compile-time, which gives the additional advantage of not impacting runtime performance. Otherwise, runtime checks are emitted to still provide correctness, but at the disadvantage of producing runtime errors instead of halting compilation, which increases development cost [35]. It would be common practice in low-level Rust crates to provide `*_unchecked()` variants for these runtime-checked methods, to give users the choice of circumventing those checks and trading performance for possible UB. Due to the goals of this work, and time constraints, this was mostly skipped.

5.4.1 Typed builder

A pattern that is often encountered in Rust is a *builder*. It tries to solve the problem of value creation, where, for big types, many values must be provided, some of which are interdependant or introduce combined constraints, and some are only available at different times.

There are multiple ways to construct a value in Rust:

- **Initializing a raw struct**: If the struct has only public fields, directly constructing it is possible. As downsides, validation of the value as a whole is not possible, since a struct can always be legally constructed from legal values of all its elements, so any checks must be performed through the types of its members. Also, staggered initialization is not possible, since at construction point, every value has to be provided. It is possible to let construction use default values for some members, but this does not equal partial initialization, since it is not fundamentally possible to tell an uninitialized value from a valid value equalling the default value of the field. Thus, this loses some type safety.
- **Constructor function**: Rust does not have constructors as language constructs, as opposed to e.g. C++. Idiomatically, constructors are member functions with the name `new()` or `new_*()`, since Rust also does not allow function overloading. Paired with lack of default parameter values, this makes writing constructors rather rigid, and is not substantially different to using raw structs. No staggered initialization is possible, as with a struct initialization, but interdependent validity checks can be performed; with the caveat, that when a type provides multiple constructors, all of them must duplicate the verification logic or use a private shared constructor that centralizes the checks. This is also manual, and can be overlooked, and the probability of oversight increases with the count of constructors.
- **Struct as constructor parameter**: An elegant combination of the two concepts above is to use a dedicated initialization struct, that is either passed to a constructor or can be cast to the target type. It acts as a sort of dictionary, or list of named parameters. This emulates named arguments and also works with private fields, since the target type construction is

hidden from the user.I It still does not solve staggered construction, for the same reasons as stated above. But one could maybe imagine using this pattern multiple times, where each initialization struct sets some parameters and generates the next stage of initialization via an opaque intermediate type, thus providing multiple phases of initialization. Implementing this from hand would be complex and error-prone, since every combination of initialized-ness has to be coded explicitly, which leads to $n!$ cases, where n equals the number of initialization parameters.

- **Typestate builder pattern**: This is where Rusts strengths come in to play. Rust provides a rich macro system, enabling the generation of the boilerplate code that is needed for such a multi-stage "build" of a value. The pattern described here is named "typestate builder" pattern, or "typed builder" pattern, and is one variant of the more general builder pattern. It annotates the target struct, generating a builder struct which allows to set every parameter by itself. Required parameters must be set exactly once, optional parameters zero or one times. This detects possible bugs as cases, where a value is forgotten to be set, or is set too many times, overwriting the previous choice. Additional checks can be added to every member and the struct as a whole, in the form of closure predicates. Syntactic sugar for flags is supported to be able to write `boolean_flag()` to enable a flag, and omission of any call to leave it of, instead of `set_boolean_flag(val: bool)`. This improves readability and provides additional correctness checks. Also, only the validity closures live in runtime; the basic correctness checks of every field being set the right amount of times are modeled within the type system, leading to compilation errors when disregarded, which is one of our goals.

  The implementation of this builder pattern creates a utility intermediate type for every possible combination of whether a type has been set. An example would be a builder for a struct `struct Point { x: f32, y: f32 }`, which represents a point on a cartesian coordinate system. A manual implementation of a builder could look like this:

```rust
struct PointWithXSet { x: f32 }
struct PointWithYSet { y: f32 }
struct PointWithNothingSet {}

impl PointWithNothingSet {
  pub fn set_x(x: f32) -> PointWithXSet { /* ... */ }
  pub fn set_y(y: f32) -> PointWithYSet { /* ... */ }
}

impl PointWithXSet {
  pub fn set_y(y: f32) -> Point { /* ... */ }
}

impl PointWithYSet {
  pub fn set_x(x: f32) -> Point { /* ... */ }
}

impl Point {
  pub fn build() -> PointWithNothingSet { /* ... */ }
}
```

Listing 9: The typed builder pattern, demonstrated on a point in the cartesian coordinate system. Non-generic implementation.

In this case, the type system makes it invalid to set an x or y coordinate multiple times, or forget to set it. There are only two possible ways to obtain a value of type `Point`:

- calling `Point::build().set_x().set_y()`
- calling `Point::build().set_y().set_x()`

Therefore we can ensure proper initialization of the point value.

Another way of modeling this pattern uses generics. Listing 10 shows such an approach.

```rust
pub struct PointBuilder<State: PointBuilderState> { state: State };

trait PointBuilderState {}
struct NothingSet;
impl PointBuilderState for NothingSet {};
struct XSet { x: f32 };
impl PointBuilderState for XSet {};
struct YSet { y: f32 };
impl PointBuilderState for YSet {};

impl Point {
  pub fn build() -> PointBuilder<NothingSet> { /* ... */ }
}

impl PointBuilder<NothingSet> {
  pub fn set_x(x: f32) -> PointBuilder<XSet> { /* ... */ }
  pub fn set_y(y: f32) -> PointBuilder<YSet> { /* ... */ }
}

impl PointBuilder<XSet> {
  pub fn set_y(y: f32) -> Point { /* ... */ }
}

impl PointBuilder<YSet> {
  pub fn set_x(x: f32) -> Point { /* ... */ }
}
```

Listing 10: The typed builder pattern, generic implementation.

On closer inspection this pattern bears resemblance of a state machine, where states are marker structs — structs with no associated data fields — that implement a marker trate — analogously, a trait without associated items —. Methods on these concrete types, which after monomorphization are the `PointBuilder` with a concrete state as type parameter, that return a `PointBuilder` with a different state type, represent transitions between those states. This is intuitive because, as a transition can only be applied to the start state and results in the end state of that transition, methods on a type can only be run on an existing value of that type, and always produce the return value.

The implementation using generics has a few advantages: Since all intermediate types are specializations of a general builder type, there can be methods on the general builder type, which correspond to transitions on any starting state.

While a typestate builder has many advantages in statical correctness, conditional branches can be difficult to handle. That is because every state of the builder is effectively a different type, and Rust does not allow items to have different types depending on a branch. This is encountered frequently when dealing with complex generics, and while staying inside this type abstraction, there is no solution besides avoiding conditionally setting fields, and instead executing the conditional code

only while calculating the value. Runtime builders do not have this issue, as every builder state has the same type, and the information of which field has been initialized is usually stored through optional types.

Because of the tradeoffs discussed between the different solutions, it can be advantageous to provide the user with multiple approaches, enabling them to choose whatever tool appropriate for the context. For example, we chose to provide both a type builder and a runtime builder for our `FileMode` struct, allowing correctness when flow of execution is well-known, and flexibility with runtime checks, when it is not.

[36]

5.4.2 `stat`

The libfuse `stat` struct is very similar to the namesake found in POSIX. Both describe an entry in an abstract filesystem, and contain most of its attributes. This set of attributes is needed for most interaction, because it provides data not limited to: the type of the entry — file, directory, symbolic link or other — it is permissions, size and modification dates. It is usually the set of information our wrapper has to provide to the surrounding system when some interaction with the filesystem takes place, e.g. listing or changing into a directory, or opening a file. [37]

Our attempt at modeling lead us to break down the struct into smaller parts, which require more attention:

- `FileType`, which is an enum flag of several possible values that have to specifically match magic IDs from the corresponding C header.
- `FilePermissions`, which are stored as a positive integer and usually displayed as an octal number in the range of `0o000` to `0o777` and represent restrictions on reading, writing and executing the underlying entry.
- three bitflags (`setuid`, `setgid`, and `vtx_flag`), that are context-dependent and enable additional features. These are stored inside the permissions integer in the underlying Unix APIs.

Other fields, like file size and modification time, were not deemed as interesting, since it can be correct for them to assume every valid bit pattern the underlying C type can represent, and checking the correctness semantically would introduce significant runtime overhead. E.g. validating modification time would have to detect modification in arbitrary files, and file size is an attribute that the wrapper has no insight into. Further insight into this problem is provided in Section 6.2.

5.4.3 `fuse_file_info`

This type, although a parameter to every operation implemented, is optional in every case, and seldomly used. In fact, due to the limited nature of our experiment, because we do not work with `open` and file descriptors, it can be assumed that the struct never be initialized. Therefore,

modeling was skipped. This does not mean that `fuse_file_info` is not a good candidate for our methodology. To the contrary: because it contains a bitset with various flags, these can easily be modeled with associated methods, which atleast provide a simple safeguard against erroneous bit operations. Additionally, some entries influence further behaviour of the filesystem and could probably be used to implement more correctness checks.

5.4.4 `FileMode`

`FileMode` is an abstraction that libfuse provides, that encapsulates both file type and permissions. To stay close to the lower level, we opted to keep this encapsulation.

This data structure is a fitting target for our builder pattern (Section 5.4.1). It is the result of a set of labels, flags and numeric values, encoded as one integer. Furthermore, each of the values encoded is useful even for a minimal filesystem, as they represent basic properties of an entry in a POSIX filesystem. Therefore, providing a builder to elegantly construct such values — as is needed e.g. inside a `getattr` implementation — will be useful for implementors of our API. We will implement two builders: one as a typestate builder, and the other as a classic runtime builder. This provides maximum flexibility for the user, since typestate builders can only be used when codepaths for initialization are static, without branching or loops. The runtime builder complements the API for those cases, at the cost of runtime overhead and less compile-time correctness checks. Both implementations will use macros to reduce boilerplate that would otherwise be significant and impractical.

Listing 11 shows the typestate builder implementation. The general pattern is to add fields to the builder struct for which builder setters should be generated, and optionally annotate them for special behavior. In our example the `setter(strip_bool)` annotation is used, which changes the signature of the setter method generated for a boolean field from `fn foo(value: bool)` to `fn foo()`, where one-time invocation corresponds to a value of `true`, and zero-time invocation to a value of `false`. This is done to increase ergonomy. The struct itself is then annotated with the `TypedBuilder` derive macro, which results in the declared struct being nothing more than a pseudo-data-structure; a schema through which to generate the resulting builder type. The `build_method(into = FileMode)` ensures that the output type of the builder is not itself, but our target type `FileMode`. Conversion is done idiomatically through Rust's `From` and `From` traits. In Rust, this is implemented via procedural macros that behave akin to compiler hooks. They get passed the annotated data structure in abstract syntax tree (AST) form and produce a token stream which replaces the annotated structure.

The runtime builder (Listing 12) implementation is similar. The `default = false` annotation creates a effect comparable to its beforementioned counterpart. If the setter on the boolean flags are not called, the `default` value of false will be assigned, making them optional. For the ergonomical part and the automatic conversion on build, no correspondence exists.

```rust
#[derive(TypedBuilder)]
#[builder(build_method(into = FileMode))]
pub struct TypedModeBuilder {
    file_type: FileType,
    permissions: FilePermissions,

    // `default = false` is implied by fallback
    #[builder(setter(strip_bool(fallback = toggle_setuid)))]
    setuid: bool,
    #[builder(setter(strip_bool(fallback = toggle_setgid)))]
    setgid: bool,
    #[builder(setter(strip_bool(fallback = toggle_vtx)))]
    vtx_flag: bool,
}
```

Listing 11: Typestate builder implementation of FileMode.

```rust
#[derive(Debug, Builder)]
pub struct RuntimeModeBuilder {
    file_type: FileType,

    permissions: FilePermissions,

    #[builder(default = false)]
    setuid: bool,
    #[builder(default = false)]
    setgid: bool,
    #[builder(default = false)]
    vtx_flag: bool,
}
```

Listing 12: Runtime builder implementation of FileMode.

The `file_type` part of the whole mode type is modelled via an enum [12, ch. 6.7]. In this case, we do not need to take advantage of the fact that Rust allowes enum variants to carry variable values of any type we might declare. A list of constants representing several choices can simply be modeled by prevalent, integer based enums like C++ [13]. The internally used integer type will, by default, be automatically be chosen by the Rust compiler for optimization headroom. Since the C API does require an unsigned 32-bit integer, we override this behavior with `#[repr(u32)]`, to not need type casts, and to make sure the enum values are guaranteed to fit.

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u32)]
pub enum FileType {
  BlockDevice = libfuse::S_IFBLK,
  CharacterDevice = libfuse::S_IFCHR,
  Fifo = libfuse::S_IFIFO,
  RegularFile = libfuse::S_IFREG,
  Directory = libfuse::S_IFDIR,
  SymbolicLink = libfuse::S_IFLNK,
  Socket = libfuse::S_IFSOCK,
}
```

Listing 13: Typestate builder implementation of FileMode.

5.4.5 `OpenFlags`

`OpenFlags` — a set of bitflags around the mode of an open file handle, like read/write access, truncating, creating — bears resemblance to (Section 5.4.4), in that we again need to model a classic C++

```rust
  /// This is repr(i32) because the target bitset (fuse_file_info::flags) and the `libc`
constants are also i32.
    #[repr(i32)]
    enum OpenFlag {
        Append = libc::O_APPEND,
        Readonly = libc::O_RDONLY,
        Writeonly = libc::O_WRONLY,
        ReadPlusWrite = libc::O_RDWR,
        Truncate = libc::O_TRUNC,
        // TODO (maybe) exhaust
    }

    pub struct OpenFlags(i32);

    impl OpenFlags {
        bitflag_accessor!(i32, append, OpenFlag::Append);
        bitflag_accessor!(i32, readonly, OpenFlag::Readonly);
        bitflag_accessor!(i32, writeonly, OpenFlag::Writeonly);
        bitflag_accessor!(i32, read_plus_write, OpenFlag::ReadPlusWrite);
        bitflag_accessor!(i32, truncate, OpenFlag::Truncate);
    }
}
```

Listing 14: OpenFlag enum .

```
macro_rules! bitflag_accessor {
    ($inner_type:ty, $name:ident, $val:path) => {
        fn $name(&self) -> bool {
            self.0 & $val as $inner_type != 0
        }
    };
}
```

Listing 15: The `bitflag_accessor!()` macro.

```
impl OpenFlags {
    fn append(&self) -> bool {
        self.0 & OpenFlag::Append as i32 != 0
    }
    fn readonly(&self) -> bool {
        self.0 & OpenFlag::Readonly as i32 != 0
    }
    fn writeonly(&self) -> bool {
        self.0 & OpenFlag::Writeonly as i32 != 0
    }
    fn read_plus_write(&self) -> bool {
        self.0 & OpenFlag::ReadPlusWrite as i32 != 0
    }
    fn truncate(&self) -> bool {
        self.0 & OpenFlag::Truncate as i32 != 0
    }
}
```

Listing 16: The expanded code from `bitflag_accessor!()`.

5.5 Error handling

While libfuse and Rust both return an error value

Chapter 6
Evaluation

To evaluate the effectiveness of our approach, we collected a sample of CVEs found in the filesystem modules of the Linux kernel in recent years. We then categorized them based on whether the approach discussed here would have been effective in preventing them. We also implemented a simple prototype filesystem using our wrapper, to provide a reference point for the ergonomy and expressiveness of the API.

6.1 CVEs

Common Vulnerabilites and Exposures / Common Vulnerability Enumerations (old) (CVEs) are standardized identifiers for publicly disclosed software vulnerabilities. Maintained by the MITRE Corporation, the CVE system provides a consistent way to reference and track security issues across tools, databases, and research. This standardization facilitates communication, comparison, and aggregation of vulnerability data.

Since our area of research centers around filesystems, we collected a sample of recent CVEs that were found inside the Linux kernel filesystem modules. A filter query (Listing 17) was crafted for the database to provide CVEs matching our criteria. From this selection, a suitable subset was extracted, focusing on CVEs where the description and metadata were complete, and the applicability of our approach could be assessed with certainty.

```
https://nvd.nist.gov/vuln/search#/nvd/home?cnaSourceIdList=386&sortOrder=5&
sortDirection=2&offset=125&rowCount=25&keyword=filesystem&cpeFilterMode=applicability&
cpeName=cpe:2.3:o:linux:linux_kernel:*:*:*:*:*:*:*:*&resultType=records
```

Listing 17: The query used to obtain a set of interesting CVEs.

Our strategy for assessing the potential success of our approach was to match the techniques, design patters and language features used in our implementation against the programming error leading to the CVE in question. In other words, the final classfications results from the question whether — if this error had occured during our development — would it fall under one of our established strategies, and would it be preventable thereof. The following table shows the result of

our evaluation, where each chosen CVE is characterized by a short description, a categorization in applicable/partially applicable/not applicable, and a rationale on our decision.

CVE ID	Problem	Solution	Score
2025-21646	the `procfs` filesystem (`procfs`) expects maximum path length of 255, this was overseen by the `afs` filesystem (`afs`) implementors, leading to a runtime error	if `procfs` API were implemented per my concept, maximum path length could be encoded in the type, so compiler could warn/error on oversight	🟢
2024-55641	When quota reservation on XFS fails due to IO errors shutting down the filesystem, inodes were mistakenly not unlocked during cleanup.	Implementing inodes in Rust can make use of the `Drop` trait, automatically unlocking them as they go out of scope[9].	🟢
2024-45003	Inside the VFS layer, during inode eviction, a race condition can result in a deadlock when inodes that are marked for deletion get accessed by a filesystem.	Rust provides no builtin solution to race conditions. While data races are automatically prevented in safe Rust, preventing deadlocks still lies in the hands of the programmer.	🔴
2022-48869	A data race in the `gadgetfs` implementation can lead to use-after-free.	This is a classic example of a multithreaded program not using the correct synchronization primitives for shared mutable state. As long as safe Rust is used for access of this state — in a hypothetical Rust-only implementation —, data races are guaranteed by the language to not occur. (However, using unsafe Rust could limit this guarantee.)	🟢/🟡
2024-42306	In the `udf` kernel module, when a corrupted block bitmap is detected, allocation is aborted but subsequent allocations will still not check the fail state and instead blindly use the allocation buffer, leading to undefined state. The solution was to use a "verified" bit to check the bitmap for validity.	This depends: if the "verified" bit existed before the issue was found, but was erroneously not checked, this can be prevented through stricter type modelling in Rust. However, if the case in question was simply not considered during implementation, and the mentioned bitflag was introduced as solution, then this constitutes a typical logic error where not all	🟡

[9] see also *RAII*

CVE ID	Problem	Solution	Score
		possible system states are considered, and cannot be prevented by our approach.	
2024-46695	A root user on an Network File System (NFS) client can, under specific circumstances, change security labels on a mounted NFS file system. This happens because a mandatory permission check was overlooked, which was documented in the contract of the function `__vfs_setxattr_noperm()`.	Our approach would allow to enforce these permission checks as part of the type system, either by a type around `__vfs_setxattr_noperm()` performing them itself, or by only yielding the correct marker types when the permissions are checked.	🟢
2025-38663	A sanity check for invalid file types was missing from the `nilfs2` module.	As concretely shown within the `FileType` enum in our wrapper library, construction of the enum from an invalid file type ID would have automatically resulted in an error.	🟢
2023-52590	Due to an interaction with VFS, renaming a directory on `ocfs2` could result in filesystem corruption, because a lock was not properly aquired.	Although, in theory, this would be part of the VFS contract that we could try to encode in the type system, this looks like a logic error as result of a complex interaction of modules. This kind bugs are notoriously hard to avoid completely by software tooling, because they require a detailed high-level understanding of components and contracts that is usually very hard to express in a machine-understandable way.	🟡/🔴
2024-50202	In `nilfs2`, errors were ignored in a procedure searching for directory entries. This could lead to a hang later on when the error are rediscovered.	Error handling is one of Rust's strengths, because they are wrapped into the return type and the language requires the programmer to give explicit instructions on how to react to the error case. This is different in C, where errors are usually hidden away in global state, or — while part of return value — there is no mechanism to force explicit handling.	🟢

CVE ID	Problem	Solution	Score
2024-47699	A potential null pointer dereference was found inside `nilfs` when dealing with a corrupted filesystem.	While unsafe Rust does not prevent accidental null pointer derefs, since we abstract away pointer access in our wrapper, and let the user deal only with native Rust owned values and references, this problem would be solved by Our libfuse wrapper	

TABLE 1. A listing of selected CVEs plus their assessment in terms of preventability

6.2 A prototype filesystem: `hello2`

To test our wrapper library, we created a minimal filesystem using it. It implements only three callbacks — `getattr`, `read`, `open` — as this is enough to provide a complete, usable filesystem [32, p. example_2hello_8c.html].

This filesystem is read-only, since that narrows down the functionality we have to implement. Files are declared in a static global array, and are even associated with a closure object, to facilitate files with dynamic content. The dynamic aspect was chosen deliberately, because it leaves less room for the wrapper to assume properties of the filesystem, which should lead to an increased ability to detect flaws. The following example (Listing 18) shows a global file table of two entries: `time.txt`, which always reads the current system date and time, and `pid.txt`, which always reads the ID of the filesystem process. This dynamic property of our test filesystem allows us increase confidence in our abstractions, by providing less stability on which to accidentally depend.

```
static FILES: LazyLock<[FileEntry; 6]> = LazyLock::new(|| {
    [
        ("/pid", Box::new(|| std::process::id().to_string())),
        (
            "/time",
            Box::new(|| format!("{}", chrono::Local::now().format("%c"))),
        ),
    // …
```

Listing 18: A global file table for our `hello2` example filesystem

Additional logic was deemed necessary to be able to build a hierarchical recursive directory data structure from the provided file table, which is needed for listing directory contents. Implementing this keeps the file table itself clean and readable, and accelerates development and testing.

Besides some boilerplate to iterate over files in a director, the only logic consists of the block `impl Filesystem for Hello2`, where we implement methods on our libfuse wrapper's filesystem trait. The implementation was straight-forward and simple, which which was one of our design goals

for our libfuse wrapper. Most low-level details and pitfalls were abstracted away. One aspect that required a disproportionally high amount of SLoC was dealing with partial and offset reads, but offloading that to the wrapper would mean that the user has to provide an array with the full file content already contained, only for the wrapper to calculate the correct offsets. This could be made possible as an additional opt-in API, but would almost certainly result in major inefficiencies, as the filesystem has to procure the whole file's content every time a partial read is requested.

Chapter 7
Conclusion

In this thesis, we explored the benefits of a strong type system regarding safety in operating system programming, by the examples of Rust and filesystems. This combination is of particular relevance, given the recent Linux kernel developments in that same direction [1], [38], and the multitude of publications regarding safe system development in Rust [4], [5], [9], [10], [23], [39], [40]. Strong type systems allow modeling contracts around APIs and data structures inside the code, which makes them automatically verifiable by static analysis. This is an improvement over documenting these as text for programmers to read and uphold, which increases cognitive load, introduces error possibilities and increases training period. We created an abstraction layer providing high-level Rust bindings to the `libfuse` C library, uplifting the types involved into carefully constructed Rust equivalents, where invariants and guarantees are compiler-verified, with a fallback on emitting automatic runtime checks where that is not possible. We collected design principles for safe system programming and methodically applied them to the chosen subset of filesystem operations necessary, while having a rich toolset for a minimal filesystem implementation. We then evaluated a sample of CVEs from the linux kernel filesystem subsystems over the past five years, assessing if — and to what degree — these vulnerabilities would have been prevented with the method we described. A minimal filesystem in Rust was created, both to test the practical viability and improve the development phase of our wrapper library, and to give a qualitative assessment of safety and ergonomy aspects of the API.

We found that our approach could in fact increase the safety of filesystems currently included in the Linux kernel. Furthermore, since most improvements were implemented as compile-time verifications, it can be assumed that their runtime impact will be nonexisting, acknowledging the need for OS routines to be maximally performant. We estimate that a substantial amount of security and safety issues could be prevented in the future by gradually moving kernel subsystems and modules to languages with strong type system guarantees.

7.1 Limitations

The current mechanism of handling errors and propagating them to libfuse is not ideal. Specifically, while we provide the API user with an idiomatic error type — Rust's `Result` — libfuse still expects

a raw `errno` be returned to signal the type of failure. A way to work around this is provided with the `Try` trait, which, when implemented on a custom type, can override the way in which error propagation on the short-circuit `?` operator is handled [31]. Implementing this trait for a custom `enum ErrnoResult<T> { Ok(T), Err(Errno) }` along with a proper conversion `impl<T> From<ErrnoResult<T>> for i32 {}` would, in theory, enable an automatic conversion, such that simply using the `?` operator on fallible fuctions inside the **trampoline functions** would correctly propagate an `i32`-encoded errno. Unfortunately, the `Try` trait is still unstable, which means its interface could change any Rust release and it is not usable while staying on the stable toolchain. Therefore, and due to time constraints, the macro solution was implemented instead.

Although one of our goals was to minimize the chance of memory faults, for which Rust brings an outstanding toolkit, the choice for using the FFI and interfacing into C code proved an obstacle to that. User code gets handed exclusively Rust-owned objects and references, where the borrow checker guarantees no memory unsafety on successful compilation. Our wrapper library, however, has to handle a large amount of pointer parameters directly from **libfuse**, where not all criteria for safe, defined memory operations can be validated. Our current model is therefore to trust **libfuse** and the pointers we are passed. Additionally, it has proven impossible to correctly generate references with appropriate lifetimes from raw pointers. As a practical solution, and to reduce the number of bugs that could have been introduced through our wrapper in using this method, we opted instead to create owned Rust objects from C parameters through copying. This introduces some overhead, although in practice the effect could be diminishingly small — memory copies are usually very fast on modern machines — and should be measured before taken into account.

Chapter 8
Future work

The beforementioned problem with memory safety and C inter-operation could have been circumvented — or at least greatly reduced — by choosing a different approach for our general architecture. Rather than implementing a wrapper for the official C library, alternatives could have been talking to the FUSE socket directly through their message protocol, or implementing our own kernel module in Rust. This was deemed out-of-scope, since the development efford would have been substantially increased.

A significant portion of the modeled types involved bounded integers, or integers that can only lie in a specific range of values, known at compile time. To this date, Rust does not include builtin types that allow such range constraints, much less some that can compile-time check their constraints. Rust has limited support for "const generics", where the generics systems allows primitive values in addition to type parameters [12]. This can be used to create support for such bounded types. Kestrer [41] have created such a crate. A provisional inspection asserted that constant constructors for the types in question are being generated, thus it is assumed that this crate would in fact bring compile-time bounded integers to our API. This could for example have been used to limit the `struct FilePermission(u16)`, to form a `struct FilePermission(BoundedU16<0, 0o777>)` or a `bounded_integer! { struct FilePermission(0, 0o777); }` which could then be instanciated with `FilePermission::new_const(0o42)`. Failure to uphold the range limits would then result in a compiler error.

Since the goal of this thesis was to evaluate the effect of modern languages with strong type systems on system programming, limiting our evaluation to filesystems creates a weak evidential basis. A future project could broaden the filter mask for CVEs, and take into consideration both other OSs and other subsystems of the Linux kernel. Selecting a greater time span would also be possible.

Panics in Rust are unfortunately not part of the type system, and methods cannot be reliably checked for panics without complete code analysis. Furthermore, while panics in general would be interesting for analyzing if a program can be crash-free, they gain significance in environments using FFI, where propagated panics can — and often will — result in UB. This lessens the protective effect of our API, since new sources of UB can be introduced by overlooking potential panic sites. For this problem, two solutions could be considered.

1. Complete manual analysis would be feasible in our case. Not many external dependencies are used, the ones that are used could be replaced, and the code footprint of our critical FFI functions is relatively small (`<100 SLoC`). A catalogue of possible panicking functions and operators would have to be compiled from The Rust Project Developers [12] and The Rust Project Developers [31], and necessary third-party crates would either have to be reviewed, or their documentation checked and trust assumptions documented.

2. One crate that tackles this problem is `no_panic` by David Tolnay, a prominent figure amongst the Rust community [42]. It provides the ability to annotate function declarations with an attribute macro, and promises to halt the compilation with an error if the function is not provably panic-free. This implies that it is possible to write functions that would not panic, but would still not compile if the compiler is unable to prove that property. The crate thereby takes a stance typical of Rust philosophy: it is preferable to reject sound programs, than to accept unsound ones. Adoption through a future project should be possible with justifiable expenditure.

Static analyzers and sanitizers could be employed to increase security beyond what is possible in pure Rust code, especially regarding unsafe Rust and foreign libraries. The unstable channel of the Rust toolchain contains options for using a variety of the renown LLVM sanitizers with Rust projects[10]. This is made easy by the fact that Rust uses LLVM as compiler backend. Alternatively, a popular tool in the Rust community is Miri[11], which can execute Rust intermediate byte inside a sandbox to detect UB. Unfortunately, as of now, it is largely unable to work with code that uses FFI. Further investigation into proper tooling is required, and could provide some potential benefits in reliability testing of our and similar solutions.

There exist concepts that extend those of Rusts typesystem and can potentially provide even more statically verified correctness, namely dependent/liquid/refinement types. One of these projects is Flux [34], [43]. With refinement types, even more constraints could be put on function arguments and return values. For example, with Flux's state today, it would be possible not only to limit integer parameters to a range, but to make this range dynamically depend on each other, and on other runtime values, while still maintaining compile-time correctness. This is achieved via converting the annotated constraints into logical predicates, which can then be algorithmically solved. Therefore, the logic solver can prove that our programs behaves correctly in every circumstance, even if the concrete runtime values are not known.

[10] https://doc.rust-lang.org/beta/unstable-book/compiler-flags/sanitizer.html
[11] https://github.com/rust-lang/miri

Chapter A
Bibliography

[1] "Rust for Linux." Accessed: Feb. 18, 2026. [Online]. Available: https://rust-for-linux.com/

[2] H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu, "An empirical study of Rust-for-Linux: The success, dissatisfaction, and compromise," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 2024, pp. 425–443.

[3] The kernel development community, "FUSE (Filesystem in Userspace) Technical Documentation." Accessed: Mar. 06, 2026. [Online]. Available: https://docs.kernel.org/next/filesystems/fuse/index.html

[4] W. Bugden and A. Alahmar, "Rust: The Programming Language for Safety and Performance." [Online]. Available: https://arxiv.org/abs/2206.05503

[5] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk, "System Programming in Rust: Beyond Safety," in *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*, in HotOS '17. Whistler, BC, Canada: Association for Computing Machinery, 2017, pp. 156–161. doi: 10.1145/3102980.3103006.

[6] C. Thompson, "How Rust Went from a Side Project to the World's Most-Loved Programming Language." Accessed: Mar. 04, 2026. [Online]. Available: https://www.technologyreview.com/2023/02/14/1067869/rust-worlds-fastest-growing-programming-language/

[7] R. Milner, "A theory of type polymorphism in programming," *Journal of Computer and System Sciences*, vol. 17, no. 3, pp. 348–375, 1978, doi: https://doi.org/10.1016/0022-0000(78)90014-4.

[8] A. Wright and M. Felleisen, "A Syntactic Approach to Type Soundness," *Information and Computation*, vol. 115, no. 1, pp. 38–94, 1994, doi: https://doi.org/10.1006/inco.1994.1093.

[9] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "Safe systems programming in Rust: The promise and the challenge," *Communications of the ACM*, vol. 64, no. 4, pp. 144–152, 2020.

[10] V. Astrauskas, P. Müller, F. Poli, and A. J. Summers, "Leveraging rust types for modular specification and verification," *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, Oct. 2019, doi: 10.1145/3360573.

[11] B. Liskov and S. Zilles, "Programming with abstract data types," *SIGPLAN Not.*, vol. 9, no. 4, pp. 50–59, Mar. 1974, doi: 10.1145/942572.807045.

[12] The Rust Project Developers, "The Rust Reference." Accessed: Feb. 18, 2026. [Online]. Available: https://doc.rust-lang.org/1.92.0/reference/

[13] cppreference.com, "cppreference.com." Accessed: Mar. 18, 2026. [Online]. Available: https://en.cppreference.com/

[14] P. Wadler, "Linear types can change the world!," in *Programming concepts and methods*, 1990, p. 5.

[15] N. Esbensen, "The cost of iterator validity," in *Proceedings of the 6th STL Workshop*, 2006, p. 34.

[16] "btrfs(5) — Linux manual page." Sep. 10, 2025.

[17] GCC Project, "Standards (Using the GNU Compiler Collection (GCC))." Accessed: Mar. 03, 2026. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Standards.html

[18] H. W. Cain and M. H. Lipasti, "Memory Ordering: A Value-Based Approach," in *Proceedings of the 31st Annual International Symposium on Computer Architecture*, in ISCA '04. München, Germany: IEEE Computer Society, 2004, p. 90.

[19] D. Howells, P. E. McKenney, W. Deacon, and P. Zijlstra, "Linux Kernel Memory Barriers." Accessed: Mar. 08, 2026. [Online]. Available: https://www.kernel.org/doc/Documentation/memory-barriers.txt

[20] Y. Tian and B. Ray, "Automatically diagnosing and repairing error handling bugs in c," in *Proceedings of the 2017 11th joint meeting on foundations of software engineering*, 2017, pp. 752–762.

[21] Linux Kernel Newbies, "Linux 2.6.14." Accessed: Mar. 06, 2026. [Online]. Available: https://kernelnewbies.org/Linux_2_6_14

[22] S. Miller *et al.*, "High velocity kernel file systems with bento," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 65–79.

[23] S. Oikawa, "The Experience of Developing a FAT File System Module in the Rust Programming Language," in *Software Engineering, Artificial Intelligence, Networking and Parallel/*

Distributed Computing, R. Lee, Ed., Cham: Springer International Publishing, 2023, pp. 45–58. doi: 10.1007/978-3-031-19604-1_4.

[24] UEFI Forum, Inc., "Unified Extensible Firmware Interface (UEFI) Specification, Release 2.11." Accessed: Mar. 03, 2026. [Online]. Available: https://uefi.org/sites/default/files/resources/UEFI_Spec_Final_2.11.pdf

[25] Microsoft, "Description of the FAT32 File System (Knowledge Base article Q154997)." Accessed: Mar. 03, 2026. [Online]. Available: https://www.uvm.edu/~jleonard/97.html

[26] Ecma International, "ECMA-107: Volume and File Structure of Disk Cartridges for Information Interchange (2nd edition)." Accessed: Mar. 03, 2026. [Online]. Available: https://www.ecma-international.org/wp-content/uploads/ECMA-107_2nd_edition_june_1995.pdf

[27] C. Berner, "fuser: Filesystem in Userspace (FUSE) for Rust." [Online]. Available: https://github.com/cberner/fuser

[28] Miguel Ojeda <ojeda@kernel.org>, "LKML: ojeda@kernel ...: [PATCH 00/13] [RFC] Rust support." Accessed: Mar. 19, 2026. [Online]. Available: https://www.ecma-international.org/wp-content/uploads/ECMA-107_2nd_edition_june_1995.pdf

[29] J. Corbet, "The (successful) end of the kernel Rust experiment." Accessed: Mar. 19, 2026. [Online]. Available: https://lwn.net/Articles/1049831/

[30] S. Klabnik, C. Nichols, C. Krycho, and Rust Community, "The Rust Programming Language." Accessed: Feb. 18, 2026. [Online]. Available: https://doc.rust-lang.org/book/

[31] The Rust Project Developers, "The Rust Standard Library." Accessed: Feb. 18, 2026. [Online]. Available: https://doc.rust-lang.org/1.92.0/std/

[32] "libfuse API documentation." Accessed: Feb. 21, 2026. [Online]. Available: https://libfuse.github.io/doxygen/

[33] "read(2) — Linux manual page." Sep. 21, 2025.

[34] N. Lehmann, A. T. Geller, N. Vazou, and R. Jhala, "Flux: Liquid types for rust," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1533–1557, 2023.

[35] B. W. Boehm, "Software engineering economics," *IEEE transactions on Software Engineering*, no. 1, pp. 4–21, 1984.

[36] I. Arye, "typed_builder: Compile-time type-checked builder derive." [Online]. Available: https://github.com/idanarye/rust-typed-builder

[37] "stat(3type) — Linux manual page." May 02, 2024.

[38] The kernel development community, “Rust.” Accessed: Mar. 11, 2026. [Online]. Available: https://docs.kernel.org/rust/index.html

[39] L. Seidel and J. Beier, “Bringing Rust to Safety-Critical Systems in Space,” in *2024 Security for Space Systems (3S)*, 2024, pp. 1–8. doi: 10.23919/3S60530.2024.10592287.

[40] I. Bang, M. Kayondo, H. Moon, and Y. Paek, “TRust: A Compilation Framework for In-process Isolation to Protect Safe Rust against Untrusted Code,” in *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA: USENIX Association, Aug. 2023, pp. 6947–6964. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/bang

[41] Kestrer, “bounded_integer: Bounded integers.” [Online]. Available: https://github.com/Kestrer/bounded-integer

[42] D. Tolnay, “no_panic: Attribute macro to require that the compiler prove a function can't ever panic..” [Online]. Available: https://github.com/dtolnay/no-panic

[43] flux-rs, “Flux: Refinement Types for Rust.” Accessed: Mar. 11, 2026. [Online]. Available: https://github.com/flux-rs/flux

Chapter B
Code listings

Chapter C
Glossary

***AST* – abstract syntax tree**: The prevalent data structure to semantically represent high-level program instructions in machine-understaandable form. ASTs are usually emitted by the parsing stage of the compiler. 33

***CVE* – Common Vulnerabilites and Exposures / Common Vulnerability Enumeration (old)**: An internationally accepted, de-facto standard, cataloguing system for vulnerabilities. In colloquial terms, "a CVE" refers to an entry in the CVE database, managed by MITRE. 4, 37, 43

***DKMS* – Dynamic Kernel Module Support**: A framework for creating Linux kernel modules and loading them at runtime, without them being compiled directly into the kernel. 10

***NFS* – Network File System** 39, 39

***OS* – Operating System** 3, 3, 3, 3, 3, 3, 3, 7, 43, 45

***POSIX* – the POSIX standard** 11, 32, 33

Rust: TODO 20, 20

***UB* – Undefined Behaviour**: TODO 1, 8, 20, 20, 27, 28, 45, 45, 46

***VFS* – Virtual File System layer**: the part of the Linux kernel that abstracts between filesystems and the rest of the system 13, 17, 38, 39, 39

`*afs*` – the `afs` filesystem: TODO 38

alignment: A property of values in memory, that specifies if their memory address is divisible by a certain power of two determined by their type's alignment requirement. It is usually a soundness violation in Rus to access unaligned values. 20

C++: The C++ programming language. 6, 7, 7, 28, 34, 35

dangling: A property of a pointer, meaning that it points to a memory address that is not accessible, or is not filled with a valid value of that pointers type. 20

***libfuse* – the libfuse C library** 11, 14, 14, 14, 14, 19, 20, 21, 22, 23, 23, 26, 26, 26, 27, 27, 28, 32, 33, 36, 43, 43, 44, 44

our libfuse wrapper 26, 26, 40, 40, 41

monomorphization: The process during Rust compilation to generate concrete non-generic functions for each instance of a generic function in the source code. Monomorphized functions don't have any generic arguments left. 31

newtype struct: TODO 24, 27

panic: A mechanism in Rust to abort the current execution of a program, often in case of an irrecoverable error. Normal control flow is interrupted and one of two behaviours can be specified: directly aborting the program, or @unwinding. 21, 21, 23, 45, 45

panic-free: A program/function/block of code that will not, under any circumstances, invoke a panic. If a block of code is panic-free, then the execution of the resulting instructions will always follow along the control flow of the written code. 21, 46

`*procfs*` – the `procfs` filesystem: TODO 38, 38

soundness: TODO define & quote 20, 20

trampoline function: A wrapper function which has the job of being a proxy for another, binary-incompatible function, setting up the correct environment and converting the call. In our case, trampoline functions are ones that have the correct ABI and signature to be passed as FUSE operations, and, when executed, redirect execution flow into Rust code providing the actual filesystem functionality. 26, 44